

PROGRAMACIÓN APLICADA · CLASE 01

ParkFlow 360

Del problema del cliente al repositorio y backlog inicial

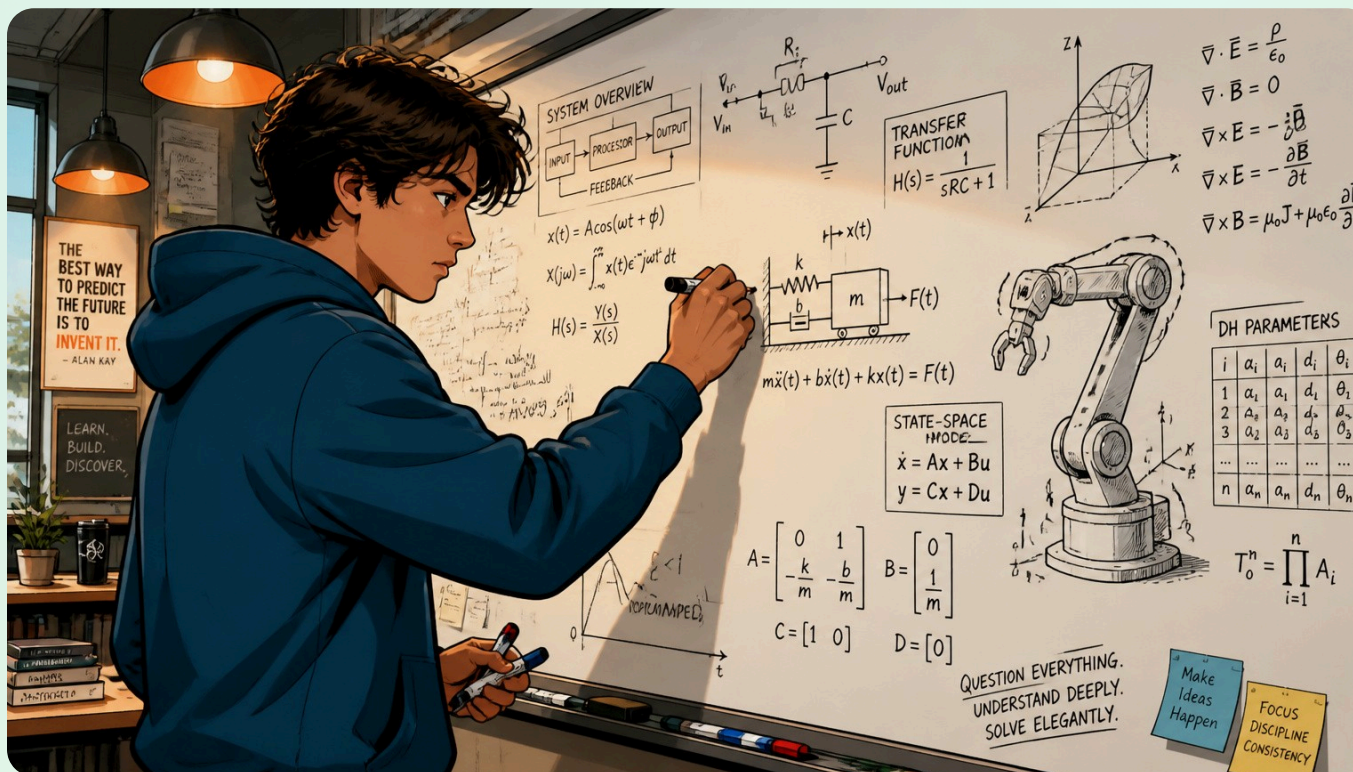
Transformar un problema ambiguo en una primera versión entendible del proyecto y dejarlo registrado en Git.



Idea clave: gana comprensión, no velocidad

Al finalizar debes poder explicar el problema, actores, alcance, exclusiones, requisitos, reglas e historias, y demostrar un repositorio con README, visión, glosario, backlog y primer commit.

- ❑ Hoy no ganas puntos por escribir código más rápido. Ganas comprensión. Si no puedes explicar qué estás construyendo, todavía no estás listo para diseñar tablas ni controllers.



Duración: 90 min · **Modalidad:**

Explicación + práctica guiada + trabajo en equipo

Herramientas: IntelliJ IDEA, PowerShell, Git, GitHub, navegador

Todavía NO usamos: Spring Boot, PostgreSQL, React, React Native

Evidencia: README + visión + glosario + backlog + commit + repositorio remoto

Qué debes ser capaz de hacer al terminar

Diferenciar conceptos

Problema, solución, actor, alcance, exclusión, RF, RN, historia y criterio de aceptación.

Leer la ficha

Sin inventar requisitos ni reducir el alcance por comodidad.

Crear repositorio Git

Desde una carpeta vacía, con documentación organizada y commit descriptivo.

Transferir el proceso

Aplicar el mismo razonamiento a tu proyecto asignado sin copiar el dominio ParkFlow.

⚠ Si puedes hacer los pasos mecánicos pero no puedes explicar por qué existen README, visión, glosario y backlog, todavía no has terminado la Clase 01.

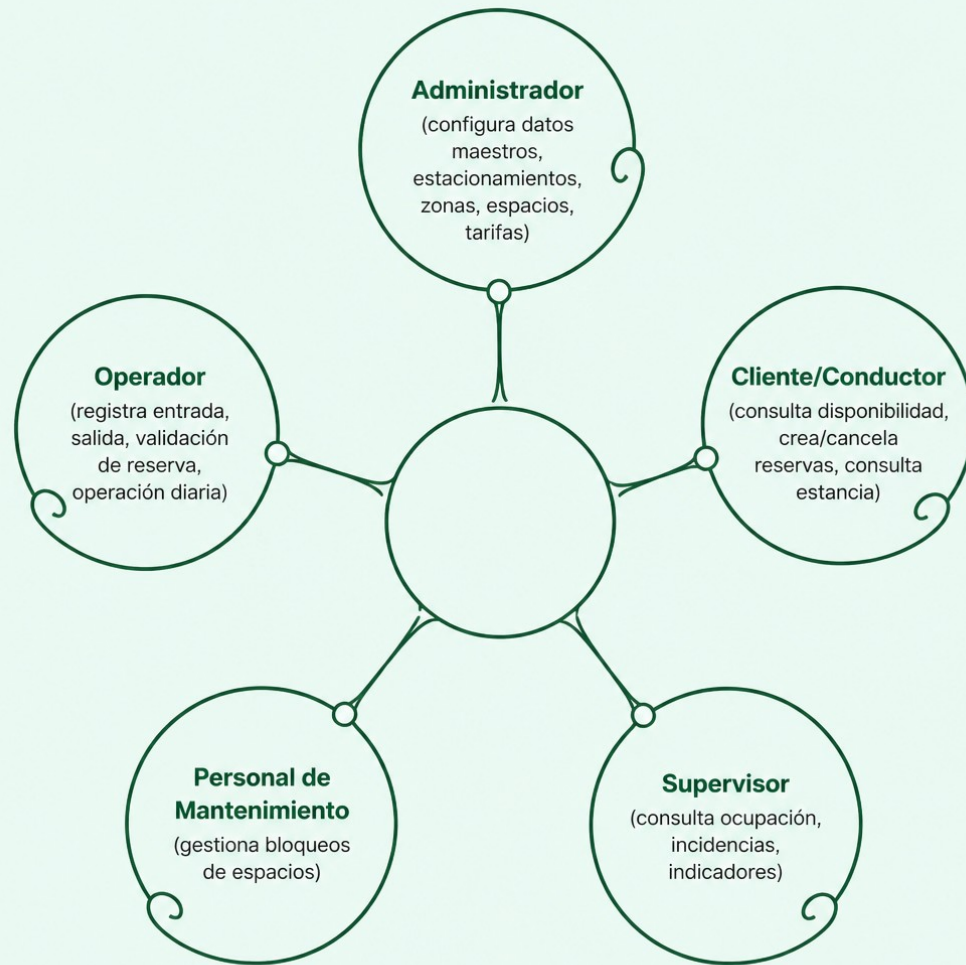


Conceptos que debes dominar antes de tocar código

Concepto	Qué significa	Ejemplo correcto	Error típico
Problema	Dolor, riesgo o ineficiencia actual	"Disponibilidad no confiable, se pierden reservas"	"Hacer una app"
Actor	Rol con interacción o responsabilidad relevante	Operador, Cliente, Supervisor	"Tabla Usuario"
Alcance	Lo que el MVP sí promete resolver	Reservas, ocupación, entradas/salidas	"Todo lo que pueda necesitar"
Exclusión	Lo que declaramos explícitamente fuera	IoT, cámaras/LPR, pasarela bancaria real	"Ya veremos después"
RF	Capacidad observable que el sistema debe ofrecer	Crear una reserva	"Usar Java 21"
Regla de negocio	Condición del dominio que gobierna una operación	Un espacio bloqueado no puede reservarse	"El botón debe ser azul"
Historia de usuario	Necesidad desde un actor y su valor	Como cliente, quiero consultar disponibilidad	"Crear tabla reservas"
Criterio de aceptación	Condición verificable que demuestra que funciona	Dado un espacio bloqueado, no debe aparecer disponible	"Funciona bien"

📌 **Regla de oro:** RF = QUÉ capacidad existe · RN = QUÉ condición debe respetar · Criterio = CÓMO comprobamos que funciona.

Comprender el problema antes de construirlo



Situación actual

La empresa opera estacionamientos urbanos usando hojas de cálculo, tickets manuales y mensajería. Resultado: inconsistencias de disponibilidad, reservas superpuestas, pérdida de trazabilidad y poca información para supervisión.

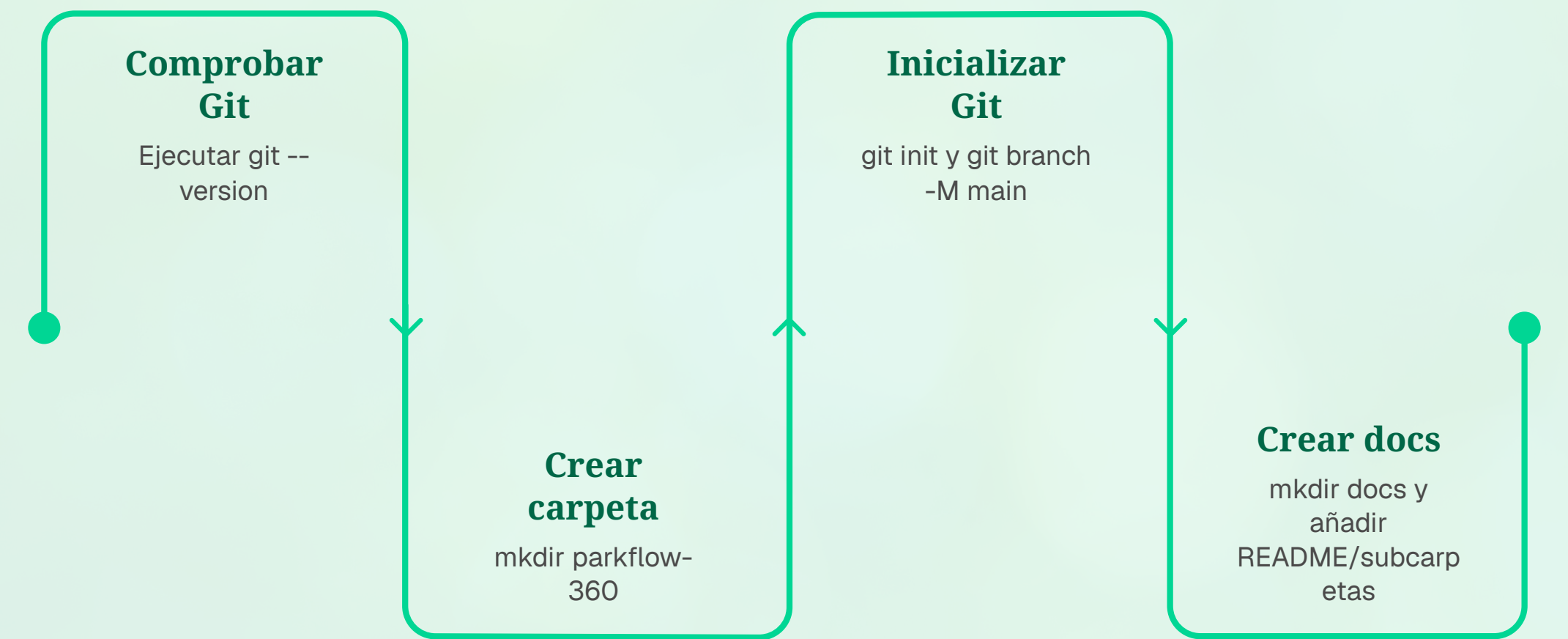
Núcleo del MVP

Estacionamientos, zonas, espacios, clientes, vehículos, disponibilidad, reservas, entrada/salida, estancias, bloqueos, incidencias e indicadores.

Fuera del MVP

IoT, cámaras/LPR, barreras físicas, GPS, pagos bancarios reales, facturación fiscal, IA para decidir reservas o precios.

Crea el repositorio desde cero



Tras crear la carpeta `parkflow-360`, la estructura documental esperada es:

```
parkflow-360
├── README.md
├── docs
│   ├── 01-vision
│   │   ├── vision-v0.1.md
│   │   └── glossary-v0.1.md
│   ├── 02-requirements
│   │   └── backlog-v0.1.md
│   └── 03-decisions
│       └── README.md
```

 **IMPORTANTE:** Hazlo desde una carpeta vacía. No descargues un ZIP "resuelto". El objetivo es comprender cada evidencia que aparece.

Contenido de los archivos y registro en Git

README.md

Problema, objetivo, actores, alcance, exclusiones, stack y estado actual.

vision-v0.1.md

Contexto, problema, objetivo, actores detallados, alcance del MVP y criterio de éxito.

glossary-v0.1.md

Términos del dominio:
Estacionamiento, Zona, Espacio, Reserva, Entrada, Estancia, Bloqueo, Incidencia...

backlog-v0.1.md

Historias priorizadas
P0/P1/P2 con formato:
Como [actor], quiero [necesidad] para [valor].

Registrar evidencia en Git

```
git status  
git add .  
git commit -m "docs: initialize ParkFlow vision and backlog"  
git log --oneline -5
```

Publicar en GitHub

```
git remote add origin  
https://github.com/USUARIO/parkflow-360.git  
git push -u origin main
```

⚠ Nunca pegues contraseñas, tokens o archivos .env en README ni los subas a Git.

TU PROYECTO

Aplica el proceso a tu proyecto asignado

El objetivo no es transformar todos los proyectos en ParkFlow. Copia el proceso de razonamiento usando exclusivamente los actores, reglas y conceptos de tu ficha oficial.

10+

Términos en glosario

Reales de tu dominio, no palabras técnicas.

8+

Historias en backlog

Priorizadas P0/P1/P2 con actor, necesidad y valor.

1

Commit por integrante

Descriptivo, con historial visible en Git.

⊗ No inventes requisitos que contradigan la ficha oficial. No reduzcas el alcance porque "parece difícil". No copies nombres ParkFlow si tu dominio usa citas, préstamos, órdenes, envíos u otros conceptos.

Errores típicos y preparación para la defensa

Errores que te hacen perder comprensión

- **Actor = Usuario:** Confunde responsabilidad con persistencia.
- **Todo es PO:** Si todo es urgente, no existe prioridad.
- **Historia = crear tabla:** Una tabla es implementación, no necesidad del actor.
- **Saltar a Spring Boot:** Termina primero la evidencia de Clase 01.

Preguntas de defensa individual

¿Qué es alcance? El compromiso de lo que el MVP sí resolverá.

¿RF y RN son lo mismo?
No. RF = capacidad. RN = condición que gobierna esa capacidad.

¿Para qué sirve el glosario? Para fijar significados compartidos del dominio.

¿Por qué aún no creamos tablas? Primero debemos identificar conceptos y reglas antes de diseñar persistencia.

Definition of Done y trabajo obligatorio

Definition of Done

01

El proyecto puede entenderse leyendo solo el README.

02

El alcance no contradice la ficha oficial y las exclusiones están escritas.

03

Existe un lenguaje inicial de dominio (glosario).

04

El backlog tiene historias priorizadas, no solo tareas técnicas.

05

Los archivos están versionados y puedes explicar git status y git log.

06

Puedes responder las preguntas de defensa sin depender de otro integrante.

Trabajo obligatorio antes de Clase 02

- Termina README v0.1, glosario y backlog inicial de tu proyecto.
- Al menos un commit por integrante.
- Relee la ficha y marca sustantivos, procesos, estados y restricciones.
- No diseñes todavía el DER ni crees tablas en PostgreSQL.
- No saltes a JPA ni controllers.
- Llega preparado para identificar entidades, atributos, relaciones y cardinalidades.



Semáforo personal: 8–10 autoevaluaciones marcadas = listo para Clase 02. Menos de 5: repite la guía desde una carpeta vacía.